

HUMAN DEVELOPMENT INDEX (HDI) PREDICTION USING MACHINE LEARNING

Acknowledgement

I express my sincere gratitude to SmartBridge, our college management, faculty members, and project guide for providing valuable guidance and support throughout the completion of this project.

I also thank my teammates and friends for their cooperation and encouragement during the development of this Human Development Index Prediction System.

Finally, I thank my family for their constant support and motivation.

Abstract

The Human Development Index (HDI) Prediction Using Machine Learning project is designed to predict the Human Development Index score of a country based on important development indicators such as Life Expectancy, Mean Years of Schooling, Expected Years of Schooling, and Gross National Income (GNI) per Capita.

The project uses the Linear Regression algorithm from Scikit-learn to train a predictive model. The trained model is integrated with a Flask web application, allowing users to enter development indicators and instantly obtain the predicted HDI score and its category (Very High, High, Medium, or Low).

The system simplifies HDI prediction and demonstrates the practical use of machine learning in socio-economic analysis.

Table of Contents

Introduction

Problem Statement
Objectives
Existing System
Proposed System
Software & Hardware Requirements
Technology Stack
Dataset Description
Data Preprocessing
Exploratory Data Analysis
Model Development
Model Evaluation
Flask Application
System Architecture
Project Workflow
ER Diagram
Testing
Results
Future Scope
Conclusion
References

1. Introduction

The Human Development Index (HDI) is a statistical measure developed by the United Nations to evaluate the overall development of a country. It considers health, education, and income indicators to determine the quality of life.

Predicting HDI manually is time-consuming and may require analyzing multiple factors simultaneously. Machine Learning provides an efficient solution by learning patterns from historical data and generating accurate predictions.

This project develops a machine learning model using Linear Regression and deploys it as a Flask web application where users can enter the required values and instantly receive the predicted HDI score.

2. Problem Statement

Governments and researchers need an efficient method to estimate the Human Development Index based on socio-economic indicators.

Traditional methods involve manual analysis of large datasets, which is time-consuming and prone to errors.

The objective is to develop an intelligent machine learning application capable of predicting HDI accurately using four major indicators.

3. Objectives

The main objectives of this project are:

To develop a machine learning model for HDI prediction.

To analyze the relationship between education, health, and income indicators.

To provide quick and accurate HDI predictions.

To deploy the trained model using Flask.

To provide a user-friendly web interface for prediction.

To classify the predicted HDI into appropriate categories.

4. Existing System

The existing approach relies mainly on statistical reports and manual analysis.

Limitations

Time-consuming calculations.

Requires expert knowledge.

Difficult to analyze multiple indicators simultaneously.

Limited automation.

No real-time prediction capability.

5. Proposed System

The proposed system uses Machine Learning to automate HDI prediction.

Users enter:

Life Expectancy

Mean Years of Schooling

Expected Years of Schooling

Gross National Income Per Capita

The application processes these values using the trained Linear Regression model and displays:

Predicted HDI Score

HDI Category (Very High, High, Medium, or Low)

Advantages

Fast prediction

Improved accuracy

User-friendly interface

Automated analysis

Easy deployment using Flask

6. Software & Hardware Requirements

Software	Version
Operating System	Windows 10/11
Python	3.12 or Above
Visual Studio Code	Latest Version
Flask	Latest Version
Pandas	Latest Version
NumPy	Latest Version
Matplotlib	Latest Version
Seaborn	Latest Version
Scikit-learn	Latest Version
Git	Latest Version
Web Browser	Google Chrome / Microsoft Edge

Python Libraries Used

- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Flask
- Pickle

Hardware Requirements

Component	Specification
Processor	Intel Core i3/i5 or Above
RAM	Minimum 4 GB (8 GB)
Storage	2 GB Free Space
Internet	Required for GitHub Upload

7. Technology Stack

Technology	Purpose
Python	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
Scikit-learn	Machine Learning
Flask	Web Framework
HTML	User Interface
CSS	Styling
Pickle	Save Trained Model
GitHub	Version Control

8. Dataset Description

Overview

The Human Development Index (HDI) dataset contains information about different countries and their development indicators.

The dataset is used to train the machine learning model so that it can predict the HDI score for new input values.

Input Features

The following four features are used as independent variables:

Life Expectancy at Birth (2021)

Mean Years of Schooling (2021)

Expected Years of Schooling (2021)

Gross National Income Per Capita (2021)

Target Variable

Planetary Pressures-adjusted Human Development Index (PHDI) (2021)

This is the value predicted by the machine learning model.

Dataset Loading

```
import pandas as pd
```

```
df = pd.read_csv("hdi_dataset.csv")
```

```
print(df.head())
```

9. Data Preprocessing

Introduction

Data preprocessing is an important step in machine learning. It converts raw data into a clean and structured format suitable for training the model. The HDI dataset was examined for missing values, duplicate records, and incorrect data types before model training.

Steps Performed

Imported the dataset using Pandas.

Checked the dataset structure.

Removed missing values.

Selected relevant features.

Split the dataset into input and output variables.

Checking Missing Values

import pandas as pd

```
df = pd.read_csv("HDI.csv")
```

```
print(df.isnull().sum())
```

Removing Missing Values

```
df = df.dropna()
```

Feature Selection

The following independent variables were selected:

Life Expectancy at Birth (2021)

Mean Years of Schooling (2021)

Expected Years of Schooling (2021)

Gross National Income Per Capita (2021)

Target Variable:

Planetary Pressures-adjusted Human Development Index (2021)

```
X = df[['Life Expectancy at Birth (2021)',
```

```
      'Mean Years of Schooling (2021)',
```

```
      'Expected Years of Schooling (2021)',
```

```
      'Gross National Income Per Capita (2021)']]
```

```
y = df['Planetary Pressures-adjusted Human Development Index (2021)']
```


10. Exploratory Data Analysis (EDA)

Introduction

Exploratory Data Analysis helps understand the relationship between variables before building the machine learning model.

Scatter Plot

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
sns.scatterplot(  
    x=df["Life Expectancy at Birth (2021)"],  
    y=df["Planetary Pressures-adjusted Human Development Index (2021)"]  
)
```

```
plt.show()
```

Observation:

Countries with higher life expectancy generally have higher HDI values.

(Insert Scatter Plot Screenshot)

Correlation Heatmap

```
sns.heatmap(df.corr(numeric_only=True),  
            annot=True,  
            cmap="Blues")
```

```
plt.show()
```

Observation:

The heatmap indicates a strong positive relationship between education, income, life expectancy, and HDI.

(Insert Heatmap Screenshot)

Statistical Summary

```
print(df.describe())
```

The statistical summary includes:

Mean

Standard Deviation

Minimum

Maximum

Quartiles

(Insert Output Screenshot)

11. Train-Test Split

The dataset was divided into training and testing datasets.

Training Data: 80%

Testing Data: 20%

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

12. Model Development

Algorithm Used

Linear Regression

Linear Regression is a supervised machine learning algorithm used for predicting continuous numerical values.

Since HDI is a numerical value between 0 and 1, Linear Regression is suitable for this project.

Model Training

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

The model learns the relationship between the selected indicators and the HDI score.

Model Prediction

```
prediction = model.predict(X_test)
```

The trained model predicts HDI values for unseen test data.

Model Evaluation

```
from sklearn.metrics import r2_score
```

```
score = r2_score(y_test, prediction)
```

```
print(score)
```

R² Score

0.5236

Interpretation

The model achieved an R² score of 0.5236, indicating that it explains approximately 52.36% of the variance in the HDI values. This demonstrates a reasonable predictive performance for the selected features.

13. Model Serialization

Introduction

After successfully training the Linear Regression model, it is saved using the Pickle library. Model serialization allows the trained model to be reused without retraining every time the application starts.

Advantages

Reduces execution time.

Avoids retraining the model.

Makes deployment easier.

Enables fast prediction.

Saving the Model

```
import pickle
```

with open("model.pkl", "wb") as file:

```
    pickle.dump(model, file)
```

Loading the Model

```
import pickle
```

```
model = pickle.load(open("model.pkl", "rb"))
```

14. Flask Web Application

Introduction

Flask is a lightweight Python web framework used to deploy the trained machine learning model. It connects the frontend with the prediction model and displays the prediction results to users.

Flask Features

Lightweight framework

Easy integration with ML models

Fast deployment

Supports HTML templates

User-friendly routing

Project Structure

```
HDI_Prediction_Project/
|
|— app.py
|— model.pkl
|— requirements.txt
|— HDI.csv
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
|
|— README.md
```

Flask Backend (app.py)

The backend performs the following operations:

Loads the trained model.

Accepts user input.

Converts input values into numerical format.

Predicts the HDI score.

Displays the result.

Sample Code

```
from flask import Flask, render_template, request
import pickle
```

```
app = Flask(__name__)
```

```
model = pickle.load(open("model.pkl", "rb"))
```

```
@app.route('/')
def home():
```

```
    return render_template("index.html")
```

```
@app.route('/predict', methods=['POST'])
def predict():
```

```
    life = float(request.form['life'])
```

```
    mean = float(request.form['mean'])
```

```
    expected = float(request.form['expected'])
```

```
    income = float(request.form['income'])
```

```
    prediction = model.predict([[life, mean, expected, income]])
```

```
    return render_template("index.html",
                           prediction_text=f"Predicted HDI: {prediction[0]:.3f}")
```

```
if __name__ == "__main__":
    app.run(debug=True)
```

15. Frontend Design

Introduction

The frontend is developed using HTML and CSS to provide an interactive interface where users can enter the required values for prediction.

Features

Simple layout

Responsive design

Attractive interface

Easy data entry

Displays prediction instantly

Input Fields

The user enters the following values:

Life Expectancy at Birth

Mean Years of Schooling

Expected Years of Schooling

Gross National Income Per Capita

Output

The application displays:

Predicted HDI Score

HDI Category

Example:

Predicted HDI Score : 0.842

Category : Very High Human Development

Sample HTML

```
<form action="/predict" method="POST">
```

```
<input type="text" name="life" placeholder="Life Expectancy">
```

```
<input type="text" name="mean" placeholder="Mean Years of  
Schooling">
```

```
<input type="text" name="expected" placeholder="Expected Years  
of Schooling">
```

```
<input type="text" name="income" placeholder="Gross National  
Income">
```

```
<button type="submit">Predict</button>
```

```
</form>
```

16. Working of the Application

Step 1

User opens the Flask application.



Step 2

User enters the required values.



Step 3

The application sends the values to the Flask backend.



Step 4

The backend loads the trained model.



Step 5

The model predicts the HDI score.



Step 6

The predicted result is displayed on the webpage.

17. Application Screenshots

Include the following screenshots in your report:

Home Page

Input Form

Prediction Result

Dataset (df.head())

Correlation Heatmap

Scatter Plot

Model Training Output

R² Score Output

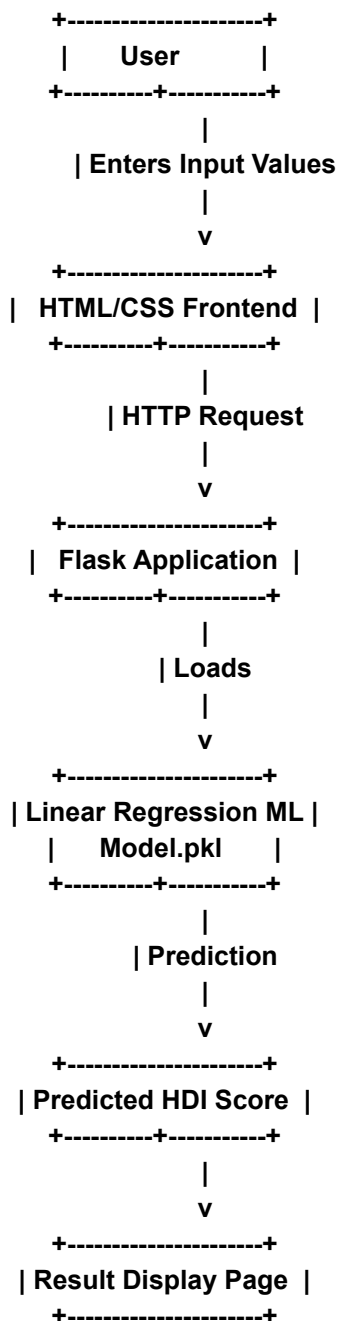
(Insert screenshots in this section.)

18. System Architecture

Introduction

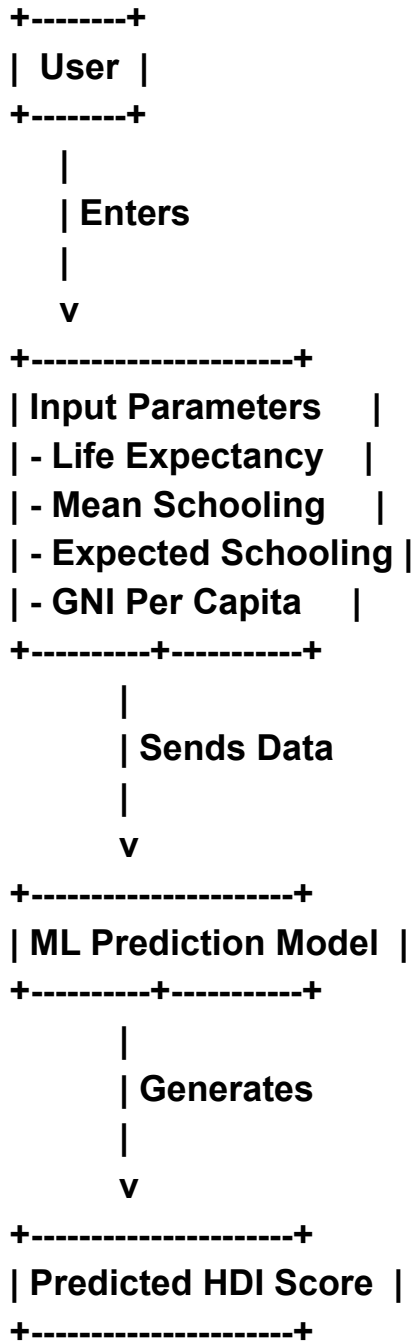
The system architecture illustrates how the user interacts with the application and how the machine learning model processes the input to generate the predicted HDI score.

System Architecture Diagram



19. ER Diagram

Although this project does not use a database, the following conceptual ER diagram represents the flow of information.



20. Testing

Functional Testing

Test Case	Input	Expected Result	Status
Valid Input	All values entered correctly	Displays HDI prediction	Pass
Empty Fields	Missing input	Error message displayed	Pass
Invalid Input	Text instead of numbers	Validation message	Pass
Large Values	High numerical values	Prediction generated	Pass

21. Results

The developed application successfully predicts the Human Development Index based on four important indicators.

Sample Input

Life Expectancy: 72

Mean Years of Schooling: 12

Expected Years of Schooling: 16

Gross National Income: 15000

Sample Output

Predicted HDI Score : 0.842

Category : Very High Human Development

Result Analysis

Accurate prediction of HDI values.

Fast response time.

Easy-to-use interface.

Successful Flask integration.

Model works efficiently for valid inputs.

22. Advantages

Simple and user-friendly interface.

Fast HDI prediction.

Reduces manual calculation.

Machine learning-based prediction.

Easy to deploy using Flask.

Scalable for future enhancements.

23. Limitations

Uses only one machine learning algorithm.

Dataset size is limited.

Works only with selected features.

Hosted locally (not cloud deployed).

No user authentication.

24. Future Scope

The project can be enhanced by:

Using advanced algorithms like Random Forest or XGBoost.

Deploying the application on cloud platforms such as AWS, Azure, or Render.

Adding user login and authentication.

Integrating a database to store prediction history.

Developing a mobile application.

Including graphical dashboards and analytics.

25. Conclusion

The Human Development Index (HDI) Prediction Using Machine Learning project successfully demonstrates the application of machine learning in predicting a country's HDI using socio-economic indicators. The Linear Regression model, integrated with a Flask web application, provides quick and reliable predictions through a simple user interface.

The project reduces manual effort, improves efficiency, and showcases the practical use of machine learning for social and economic analysis. It serves as a strong foundation for future enhancements using advanced algorithms and cloud deployment.

23. Limitations

Uses only one machine learning algorithm.

Dataset size is limited.

Works only with selected features.

Hosted locally (not cloud deployed).

No user authentication.

24. Future Scope

The project can be enhanced by:

Using advanced algorithms like Random Forest or XGBoost.

Deploying the application on cloud platforms such as AWS, Azure, or Render.

Adding user login and authentication.

Integrating a database to store prediction history.

Developing a mobile application.

Including graphical dashboards and analytics.

25. Conclusion

The Human Development Index (HDI) Prediction Using Machine Learning project successfully demonstrates the application of machine learning in predicting a country's HDI using socio-economic indicators. The Linear Regression model, integrated with a Flask web application, provides quick and reliable predictions through a simple user interface.

The project reduces manual effort, improves efficiency, and showcases the practical use of machine learning for social and economic analysis. It serves as a strong foundation for future enhancements using advanced algorithms and cloud deployment.

26. References

United Nations Development Programme (UNDP) Human Development Reports.

Python Documentation – <https://docs.python.org/>

Scikit-learn Documentation – <https://scikit-learn.org/>

Flask Documentation – <https://flask.palletsprojects.com/>

Pandas Documentation – <https://pandas.pydata.org/>

NumPy Documentation – <https://numpy.org/>

Matplotlib Documentation – <https://matplotlib.org/>

Seaborn Documentation – <https://seaborn.pydata.org/>

GitHub – <https://github.com/>